Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»



Кафедра 09.02.07

ПРАКТИЧЕСКАЯ РАБОТА № 4

Дисциплина: МДК 03.01

Тема: Версионирование REST API и обратная совместимость

Вариант № 2

Студент: Шваб Роберт Эдуардович Подпись _____

Группа: И13_24_11

Руководитель: _____ Подпись _____

Дата представления: «___» _____ 2026 г.

Оценка: _____

Санкт-Петербург 2026

СОДЕРЖАНИЕ

1 ВЫПОЛНЕНИЕ ПРАКТИЧЕСКОЙ РАБОТЫ....3
1.1 Цель и исходные данные....3
1.2 Контракты API....3
1.3 Проверка старого клиента....4
1.4 Программа и воспроизведение результата....4
1.5 Выводы....4
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ....5
ПРИЛОЖЕНИЕ А....6
ПРИЛОЖЕНИЕ Б....9

1 ВЫПОЛНЕНИЕ ПРАКТИЧЕСКОЙ РАБОТЫ

1.1 Цель и исходные данные

Цель работы — реализовать два контракта REST API на ASP.NET Core и проверить, что после добавления поля priority старый клиент продолжает работать.

Постановка работы приведена в перечне заданий МДК 03.01 [1].

Исходники и отчёты: репозиторий MDK0301-ServiceDesk на GitHub

1.2 Контракты API

Маршрутизация ASP.NET Core [2] разделяет контракты по URL. v1 отдаёт id, title и status; v2 возвращает те же поля и priority. Оба маршрута читают одну запись в TicketStore. В учебном стенде записи сохраняются на время работы процесса.

При регистрации через v1 приоритет по умолчанию равен 3; при создании через v2 принимаются значения от 1 до 4. Неизвестный идентификатор даёт HTTP 404. Маршруты перечислены в таблице 1; схема двух контрактов показана на рисунке 1.

Таблица 1 — Маршруты и назначение методов API

Метод	Маршрут	Назначение
GET	/health	Состояние API
GET, POST	/api/v1/tickets	Старый контракт
GET, POST	/api/v2/tickets	Новый контракт
GET	/api/v1/tickets/{id}	Карточка без priority
GET	/api/v2/tickets/{id}	Карточка с priority
POST	/api/v2/sync	Повторяемая отправка события

Источник: составлено по материалам практической работы.

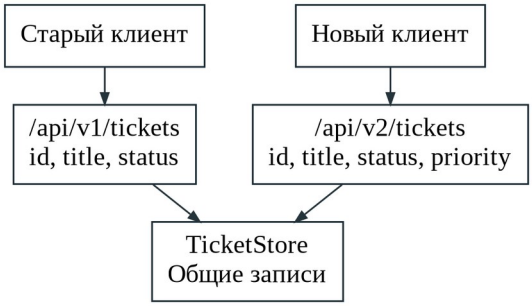


Рисунок 1 — Маршруты v1 и v2 к общему хранилищу

1.3 Проверка старого клиента

Клиент создаёт заявку через v1, считывает типизированную карточку и проверяет точный набор полей. Затем через v2 читает ту же запись и проверяет приоритет. Дополнительно проверяются 404, недопустимый приоритет и повторная доставка одного события.

Фактический вывод программы приведён в приложении Б. Запуск из Visual Studio: проект Verification, профиль ПР 4, Ctrl+F5.

1.4 Программа и воспроизведение результата

REST API реализован в Code/ServiceDeskApi/Program.cs. Старый клиент проверяется в Code/ApiCompatibilityTests/Program.cs. Эти файлы включены в решение и в приложение А; для показа используется профиль «ПР 4».

Перед показом выполняется «Сборка → Пересобрать решение». Результат сборки виден в окне «Вывод → Сборка», а результат программы — в отдельной консоли Visual Studio.

1.5 Выводы

Тесты подтверждают совместимость старой карточки и корректное добавление поля в v2. Исходники сервера и клиента приведены в приложении А.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 МДК 03.01. Перечень практических работ № 1–8. Учебное задание.

2 Microsoft Learn. ASP.NET Core documentation. https://learn.microsoft.com/aspnet/core/

ПРИЛОЖЕНИЕ А

Исходный код программы

Code/ServiceDeskApi/Program.cs

```
using System.Collections.Concurrent;
using Microsoft.AspNetCore.Http.HttpResults;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddSingleton<TicketStore>();
var app = builder.Build();
app.MapGet("/health", () => Results.Ok(new { status = "ok", version = "1.0.0" }));
// V1 retains original response contract for legacy clients.
app.MapGet("/api/v1/tickets", (TicketStore db) => Results.Ok(db.All().Select(t => new TicketV1(t.Id, t.Title,
t.Status))));
app.MapGet("/api/v1/tickets/{id:int}", (int id, TicketStore db) => db.Get(id) is { } t ? Results.Ok(new
TicketV1(t.Id,t.Title,t.Status)) : Results.NotFound());
app.MapPost("/api/v1/tickets", (NewTicket request, TicketStore db) =>
{
   if (string.IsNullOrWhiteSpace(request.Title)) return Results.BadRequest("Title is required");
   var t = db.Add(request.Title.Trim());
   return Results.Created($"/api/v1/tickets/{t.Id}", new TicketV1(t.Id,t.Title,t.Status));
});
// V2 extends response with Priority; V1 endpoints are unchanged.
app.MapGet("/api/v2/tickets", (TicketStore db) => Results.Ok(db.All().Select(t => new
TicketV2(t.Id,t.Title,t.Status,t.Priority))));
app.MapGet("/api/v2/tickets/{id:int}", (int id, TicketStore db) => db.Get(id) is { } t ? Results.Ok(new
TicketV2(t.Id,t.Title,t.Status,t.Priority)) : Results.NotFound());
app.MapPost("/api/v2/tickets", (NewTicketV2 request, TicketStore db) =>
{
   if (string.IsNullOrWhiteSpace(request.Title) || request.Priority is < 1 or > 4)
      return Results.BadRequest("Title and priority 1..4 are required");
   var t = db.Add(request.Title.Trim(), request.Priority);
   return Results.Created($"/api/v2/tickets/{t.Id}", new TicketV2(t.Id,t.Title,t.Status,t.Priority));
});
// The idempotency key identifies a client event within this running service.
app.MapPost("/api/v2/sync", (OfflineTicket request, TicketStore db) =>
{
   if (request.EventId == Guid.Empty || string.IsNullOrWhiteSpace(request.Title))
      return Results.BadRequest("EventId and title are required");
   return Results.Ok(db.Sync(request.EventId, request.Title.Trim()));
});
app.MapGet("/", () => Results.Content("""
<!doctype html><html lang="ru"><meta charset="utf-8">
<title>Служба заявок — API</title>
<style>body{font:20px Arial;margin:45px;color:#19314a;background:#f6f8fa}
pre{background:white;padding:16px;border:1px solid #ccd6de;font:17px monospace}
h1{font-size:33px}h2{font-size:22px}</style>
<h1>Служба заявок</h1><p>МДК 03.01 · ПР 4 · Ответы работающего REST API</p>
<div id="results"></div>
<script>
(async () => {
 for (const path of ['/health','/api/v1/tickets','/api/v2/tickets']) {
   const response = await fetch(path);
   const title = document.createElement('h2');
   title.textContent = 'GET ' + path + ' — HTTP ' + response.status;
   const result = document.createElement('pre');
```

```
  result.textContent = JSON.stringify(await response.json(),null,2);
  document.querySelector('#results').append(title,result);
 }
})();
</script></html>
""", "text/html; charset=utf-8"));
app.Run();
public partial class Program { }
record OfflineTicket(Guid EventId, string Title);
record NewTicket(string Title);
record NewTicketV2(string Title, int Priority);
record TicketV1(int Id, string Title, string Status);
record TicketV2(int Id, string Title, string Status, int Priority);
record Ticket(int Id, string Title, string Status, int Priority);
class TicketStore
{
   private readonly ConcurrentDictionary<int,Ticket> _items=new();
   private int _next=0;
   private readonly Dictionary<Guid, Ticket> _synced = new();
   public Ticket Sync(Guid eventId, string title)
   {
      lock (_synced)
      {
         if (!_synced.TryGetValue(eventId, out var ticket))
            _synced.Add(eventId, ticket = Add(title));
         return ticket;
      }
   }
   public IEnumerable<Ticket> All()=>_items.Values.OrderBy(x=>x.Id);
   public Ticket? Get(int id)=>_items.TryGetValue(id,out var t)?t:null;
   public Ticket Add(string title,int priority=3)
   {
      var t=new Ticket(Interlocked.Increment(ref _next),title,"Open",priority);
      _items[t.Id]=t; return t;
   }
}
```

Code/ServiceDeskApi/ServiceDeskApi.csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><ImplicitUsings>enable</
ImplicitUsings><Nullable>enable</Nullable></PropertyGroup>
</Project>
```

Code/ApiCompatibilityTests/Program.cs

```
using System.Net;
using System.Net.Http.Json;
using System.Text.Json;

var address = args.FirstOrDefault() ?? "http://127.0.0.1:5000";
using var http = new HttpClient { BaseAddress = new Uri(address) };
var created = await http.PostAsJsonAsync("/api/v1/tickets", new { title = "Проверить принтер" });
Assert(created.StatusCode == HttpStatusCode.Created, "POST v1 returns 201");
var ticket = await created.Content.ReadFromJsonAsync<OldTicket>();
Assert(ticket is { Id: > 0, Title: "Проверить принтер", Status: "Open" }, "old client deserializes v1");
var id = ticket!.Id;
using var v1 = JsonDocument.Parse(await http.GetStringAsync($"/api/v1/tickets/{id}"));
Assert(v1.RootElement.EnumerateObject().Select(p => p.Name).Order()
   .SequenceEqual(new[] { "id", "status", "title" }), "v1 field set is unchanged");
using var v2 = JsonDocument.Parse(await http.GetStringAsync($"/api/v2/tickets/{id}"));
```

```
Assert(v2.RootElement.GetProperty("priority").GetInt32() == 3, "v2 adds default priority");
Assert((await http.GetAsync("/api/v1/tickets/2147483647")).StatusCode == HttpStatusCode.NotFound, "missing
ticket returns 404");
Assert((await http.PostAsJsonAsync("/api/v2/tickets", new { title = "test", priority = 9 })).StatusCode
    == HttpStatusCode.BadRequest, "v2 rejects invalid priority");
var key = Guid.NewGuid();
var item = new { eventId = key, title = "Тест повторной доставки" };
var first = await http.PostAsJsonAsync("/api/v2/sync", item);
var second = await http.PostAsJsonAsync("/api/v2/sync", item);
first.EnsureSuccessStatusCode(); second.EnsureSuccessStatusCode();
Assert((await first.Content.ReadFromJsonAsync<OldTicket>())!.Id ==
    (await second.Content.ReadFromJsonAsync<OldTicket>())!.Id, "duplicate event keeps the same ticket ID");
static void Assert(bool passed, string label)
{
    if (!passed) throw new InvalidOperationException("FAIL: " + label);
    Console.WriteLine("PASS: " + label);
}
record OldTicket(int Id, string Title, string Status);
```

Code/ApiCompatibilityTests/ApiCompatibilityTests.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
<PropertyGroup><OutputType>Exe</OutputType><TargetFramework>net8.0</
TargetFramework><ImplicitUsings>enable</ImplicitUsings><Nullable>enable</Nullable></PropertyGroup>
</Project>
```

ПРИЛОЖЕНИЕ Б

Фактический вывод контрольного прогона

```
PASS: POST v1 returns 201
PASS: old client deserializes v1
PASS: v1 field set is unchanged
PASS: v2 adds default priority
PASS: missing ticket returns 404
PASS: v2 rejects invalid priority
PASS: duplicate event keeps the same ticket ID
```

Снимки выполнения программы в Visual Studio

Я запустил программы в Visual Studio под Windows. На снимках приведены исходный код и результаты контрольного запуска.

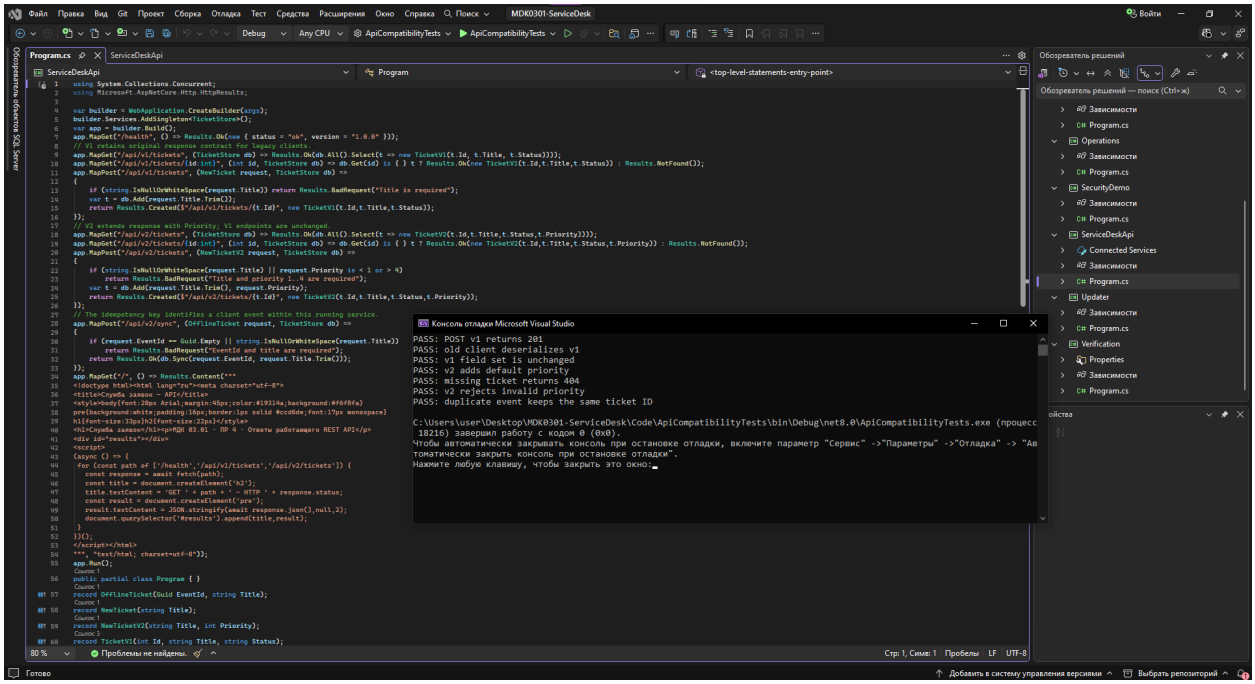


Рисунок Б.1 — Проверка обратной совместимости API в Visual Studio. Все семь проверок завершились успешно.